ASSIGNMENT #4 — FACADE PATTERN

Course: ShP-2216 – Software Design Patterns (OP 6B06102)

Name: Smat Zhangir

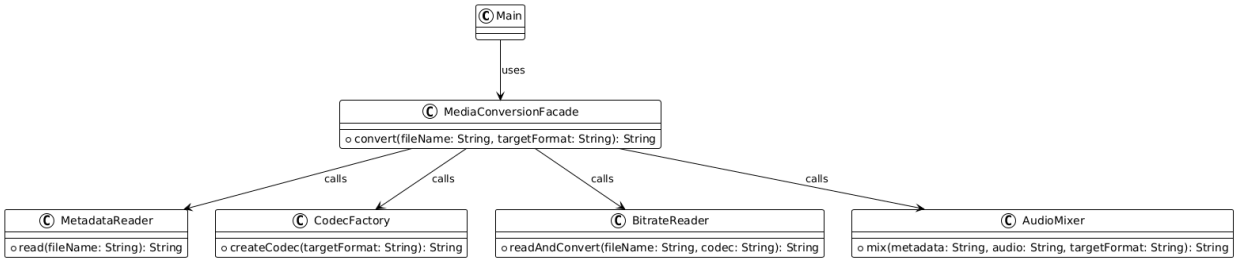
Group: SE-2522

Topic: Option A

1. Introduction

This project simulates media file conversion. Conversion involves several steps, including reading file metadata, selecting a codec, converting the audio data, and preparing the output. Calling every subsystem class from the client would make the client more complicated. MediaConversionFacade provides one method, convert, to coordinate these steps.

2. UML Class Diagram



3. Clean Code

1. Meaningful names

convert, read, createCodec, and readAndConvert describe what each method does.

2. Single Responsibility

MediaConversionFacade coordinates the workflow. MetadataReader reads metadata, CodecFactory validates and selects the codec, BitrateReader simulates conversion, and AudioMixer prepares the result. The facade does not implement each subsystem's work itself.

3. Simple public API

The client calls convert(fileName, targetFormat) instead of creating and coordinating four subsystem objects. The facade does not expose those objects as part of its public API.

4. Small, focused classes

Each subsystem class has one main responsibility and a small public interface.

5. Error handling

convert rejects empty file names and empty target formats. CodecFactory rejects unsupported formats with an IllegalArgumentException, preventing an invalid conversion request from continuing.

4. Conclusion

The Facade pattern gives the client one simple entry point for a multi-step operation. This example hides the order of subsystem calls and the details of metadata reading, codec selection, conversion, and mixing.

5. GitHub Repository

https://github.com/smatD/assignment4